[100 pts] Part 2: TCP] **(5 pts progress check in lab)**

In this section, we explore several aspects of the TCP protocol. First, we will upload a file to a webserver and observe the flow of packets over the TCP connection. Next, we will download a large file and introduce losses on the link to observe TCP's reliable data delivery service in action. You will be TCP experts by the end!

A. Upload a Great Story to a Web Server

In this section, you will observe the packets in flight while TCP uploads a text file.

Environment: Use the **VIRTUAL MACHINE in LAB** for these problems. We can not guarantee any other environment will work.

Follow these instructions in the order they are given -

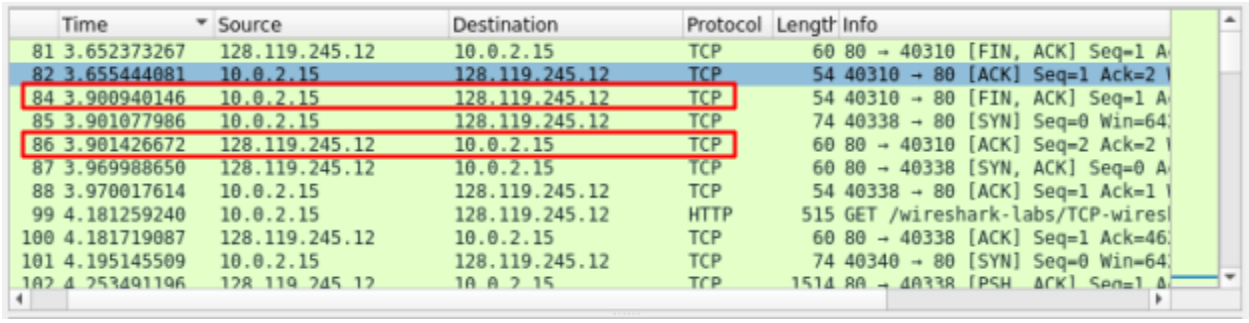
- Download the ASCII copy of Alice in Wonderland http://gaia.cs.umass.edu/wiresharklabs/alice.txt and store the file on your computer. (Read if you have time- it's a great story!)
- Start Wireshark and begin packet capture - this capture is used for Q6 thru Q8
- Load the page http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html
- Upload the alice.txt file to the webpage. Once the file has been uploaded, a short congratulations message will be displayed in your browser window. End the Wireshark capture.

TCP Basics

Referring to the Alice in Wonderland upload, answer the following questions for the TCP segments you observe in Wireshark:

Question 6 [15 pts]: HTTP and TCP Connection Setup

a. Observe the TCP connection handshake in Wireshark. Based on what you see, calculate the RTT between your client and the server. Show calculations and include **a screenshot** with circled information used to determine the RTT.



	Time	Source	Destination	Protocol	Length	Info
81	3.652373267	128.119.245.12	10.0.2.15	TCP	60	80 → 40310 [FIN, ACK] Seq=1 A
82	3.655444081	10.0.2.15	128.119.245.12	TCP	54	40310 → 80 [ACK] Seq=1 Ack=2
84	3.900940146	10.0.2.15	128.119.245.12	TCP	54	40310 → 80 [FIN, ACK] Seq=1 A
85	3.901077986	10.0.2.15	128.119.245.12	TCP	74	40338 → 80 [SYN] Seq=0 Win=64
86	3.901426672	128.119.245.12	10.0.2.15	TCP	60	80 → 40310 [ACK] Seq=2 Ack=2
87	3.969988650	128.119.245.12	10.0.2.15	TCP	60	80 → 40338 [SYN, ACK] Seq=0 A
88	3.970017614	10.0.2.15	128.119.245.12	TCP	54	40338 → 80 [ACK] Seq=1 Ack=1
99	4.181259240	10.0.2.15	128.119.245.12	HTTP	515	GET /wireshark-labs/TCP-wires
100	4.181719087	128.119.245.12	10.0.2.15	TCP	60	80 → 40338 [ACK] Seq=1 Ack=46
101	4.195145509	10.0.2.15	128.119.245.12	TCP	74	40340 → 80 [SYN] Seq=0 Win=64
102	4.253491196	128.119.245.12	10.0.2.15	TCP	1514	80 → 40338 [PSH, ACK] Seq=1 A

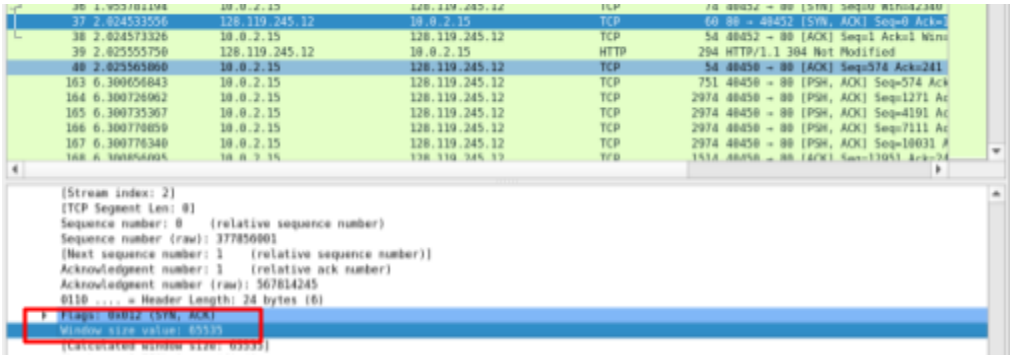
The first SYN packet is sent by the client at timestamp 3.900940146 (packet 84). The SYN-ACK is sent by the server to the client at timestamp 3.901426672 (packet 86). Hence, the **RTT** is 3.901426672−3.900940146 = **0.000486526 seconds (or 0.49 ms).**

b. <u>Sequence Numbers and Receiver's Window Size:</u>

- Explain why the initial SYN packet must include a sequence number and describe its continued role in the TCP connection..
  The SYN packet includes an initial sequence number because it helps establish the starting point for the data flow between the client and server. Each subsequent segment in the TCP stream will use this initial sequence number to calculate its position in the byte stream, allowing both sender and receiver to keep track of received data.

  During the connection, each subsequent packet's sequence number increments based on the amount of data sent, ensuring ordered delivery and enabling retransmission in case of packet loss.

- What is the window size returned by the Server according to Wireshark? Explain the purpose of the window size number and highlight it in a **screenshot.**
  The window size number controls the flow of data, allowing the sender to know how much data can be sent before requiring acknowledgement, thus helping to manage network congestion.
  The window size returned is 65535 bytes



c. <u>SYN-ACK Segment Details:</u>

- Why are the sequence numbers different in the SYN and SYN-ACK packets?
  The sequence number in the SYN packet is generated by the client and is used as the initial sequence number for that side of the connection. When the server responds with a SYN-ACK, it includes its own sequence number (initial sequence number for the server side) to establish its starting point in the connection. This ensures bidirectional data integrity and reliability in the TCP connection.

- Explain how the acknowledgment number is calculated in the SYN-ACK packet.
  The acknowledgment number in the SYN-ACK packet is calculated by adding 1 to the initial sequence number of the SYN packet sent by the client. This indicates that the server has successfully received the SYN packet and is expecting the next byte of data to start at this sequence number. This mechanism ensures proper synchronization and acknowledgment in the TCP connection handshake.

Ready to Send! – TCP File Upload

Now that the handshake is complete, we can look at the data segments.

<u>Question 7 [14 pts]:</u><u>The File Upload</u>:

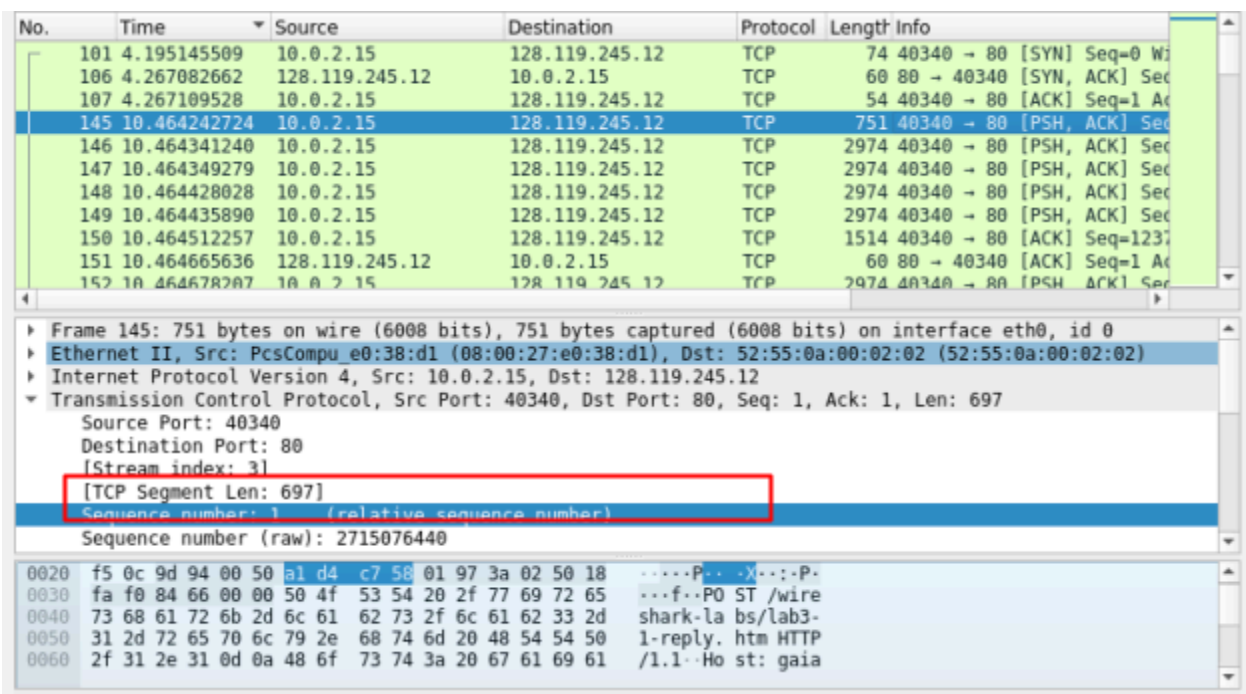
<u>Hunting for the TCP Payload (a) and (b):</u>

a. Find the first TCP segment that carries the HTTP message for the file upload in (Q6a). Convince yourself that the payload of this packet is the HTTP request message by analyzing this TCP packet. Find the bytes of the payload as demonstrated in lab. Highlight these values in a **screenshot**.

   Does it surprise you that this message is carried over TCP? What is the TCP **segment length** and **relative** sequence number of this TCP segment?

   The TCP Segment length is 697 bytes. The relative sequence number is 1. This is the first payload segment because if we follow the TCP stream, it contains the POST http message.

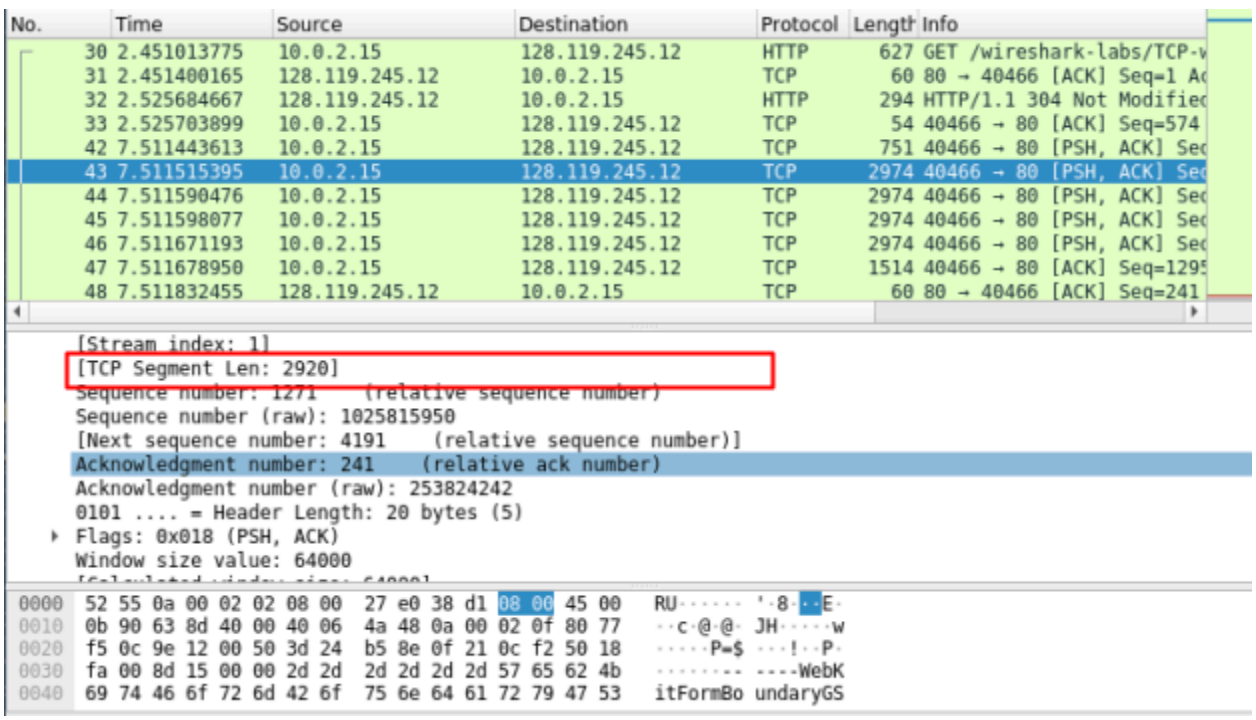
   It's not surprising that the HTTP request message is carried over TCP because TCP provides reliable data transfer, which is essential for file uploads where data integrity is critical. TCP ensures ordered delivery, retransmission in case of packet loss, and flow control.

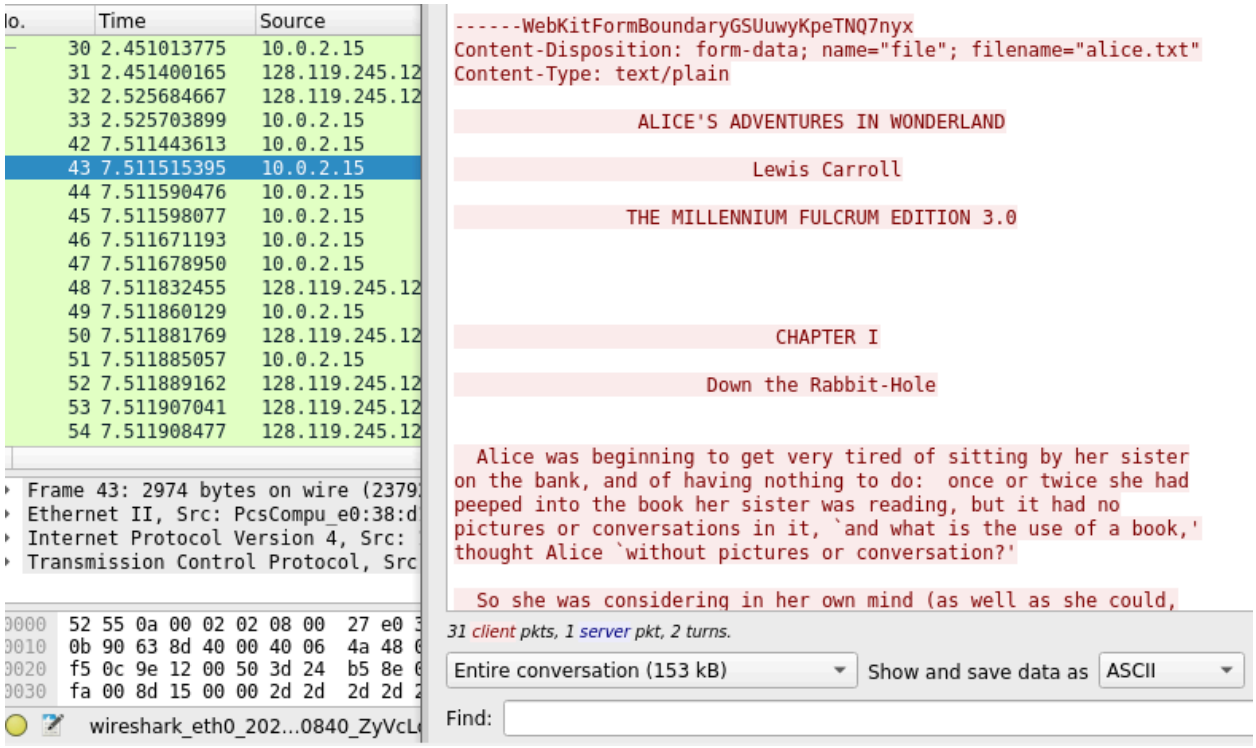


b. Next comes the data! Find the first TCP segment carrying data –the part of the file transfer that carries the initial bytes of the actual Alice in Wonderland text. Convince yourself that the payload of this TCP segment is part of the Alice in Wonderland file (find the bytes of the payload as demonstrated in the lab).

What is the TCP **segment length** and **relative** sequence number of this TCP segment? Why is it larger than the TCP segment in (a)?
Screenshot: Demonstrate your findings in a **screenshot** that highlights the data that you see (visible in the payload bytes) and also the segment length and relative sequence number of the segment, highlighted.
The TCP segment length is 2920 bytes
It is larger than the initial TCP segment because the payload begins to carry the actual file data in alice.txt

(note: mininet crashed, so I had to open wireshark again. The numbers might be a little different)





c. Look at the TCP **segment length** and **relative** sequence numbers for several subsequent segments (after the one in (b)). Can you discern a pattern? Explain how sequence numbers are calculated in subsequent segments and what they represent in the context of the TCP stream.

In subsequent TCP segments, the sequence numbers increase by the length of the payload in the previous segment. Each sequence number represents the byte offset of the first payload byte in the segment relative to the initial sequence number. This ensures that all bytes in the data stream are accounted for, allowing the receiver to reassemble data in the correct order, even if packets arrive out of sequence.

After some time, the sequence number drops drastically, indicating that there was a TCP segment loss or out of order delivery. After the drop, the subsequent TCP segments increase the length of the payload again. This pattern repeats throughout the TCP stream.

The pattern reflects TCP's reliability and ordered delivery, critical for transferring large files like alice.txt.

Question 8 [9 pts]: Graphing the RTT (Round Trip Time):
Wireshark has a nice feature that allows you to plot the RTT for each of the transmitted TCP segments. Click on a TCP segment that is being sent from the client to the gaia.cs.umass.edu server.
Navigate to: Statistics -> TCP Stream Graphs -> Round Trip Time to visualize the RTT data.
Note: The direction of the transfer is important! Switch direction until you display measurements of the transmitted segments.

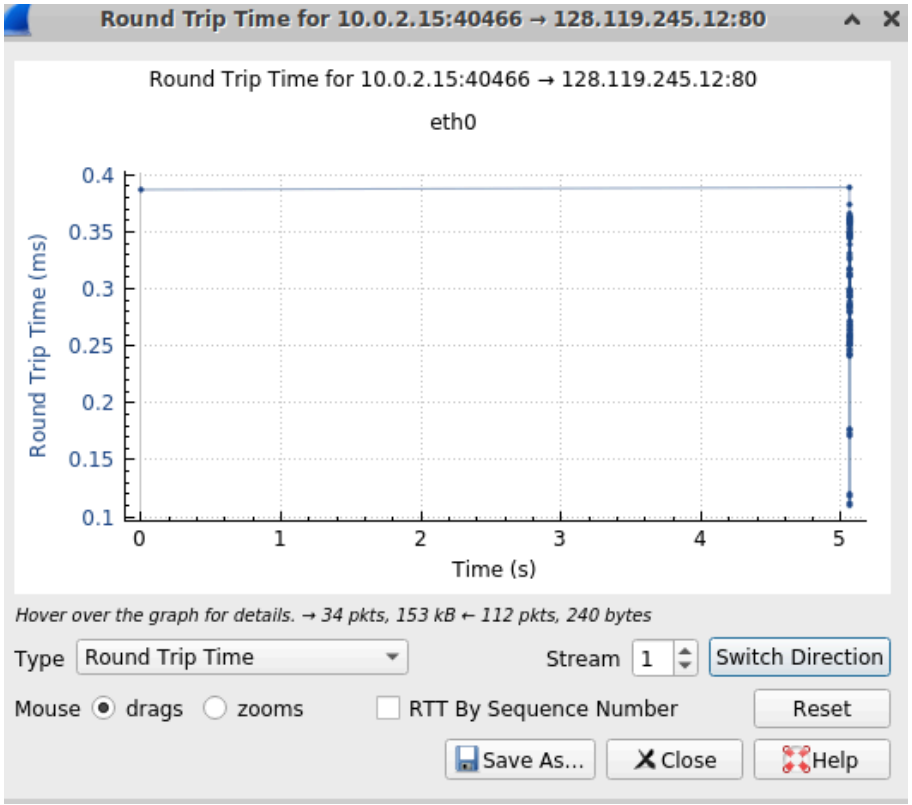
a. How would the values shown in this graph be used as a part of TCP's retransmission timeout (RTO) calculation?

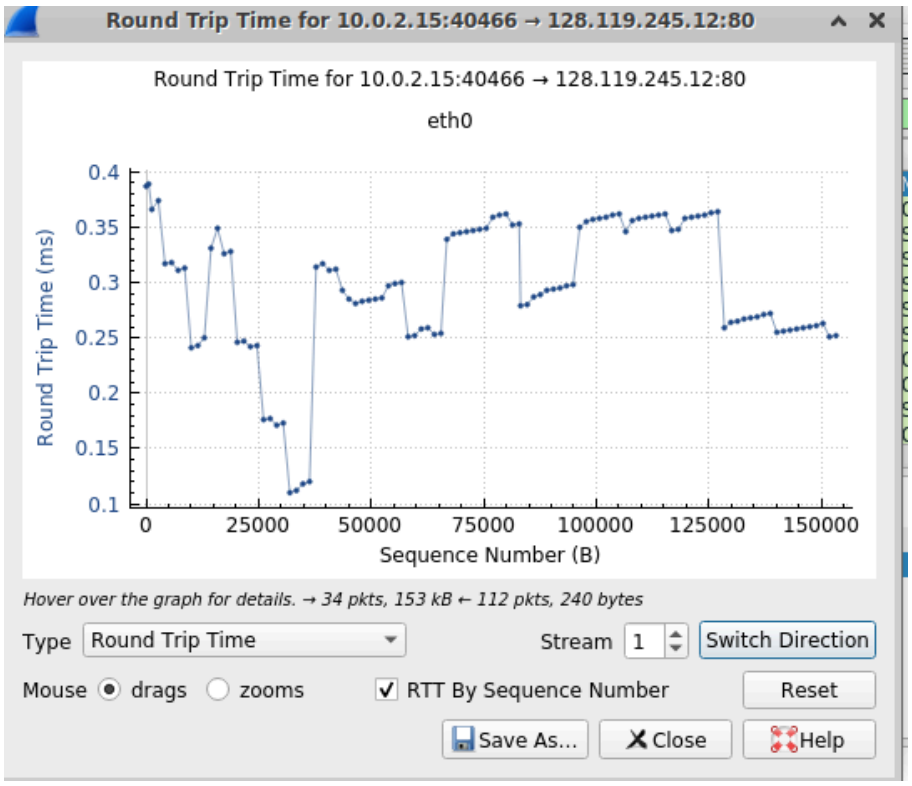
   The graph helps visualize how RTT evolves over time, indicating whether the network is stable or experiencing delays. The RTO can be calculated as RTO = (estimated RTT) + 4 * RTTVAR (where RTTVAR is the RTT variance (aka. DEVRTT)).

b. Discuss your observations in your RTT Graph. (Think about the discussion in class related to TCP's RTO calculation and the graph that was discussed. Chapter 3 is also a good reference.)

   Include a **screenshot** of this graph.

   Initially, there are higher RTT values, likely due to the connection setup and path stabilization during the TCP handshake. Over time, the RTT stabilizes around a lower value, indicating consistent network performance. Occasional spikes and drops in RTT may result from network congestion, retransmissions, or variations in queuing delay. These fluctuations highlight TCP's adaptive nature, as it uses RTT measurements to dynamically adjust the retransmission timeout (RTO), ensuring reliability and efficiency. As sequence numbers progress, the RTT becomes more consistent, reflecting smoother data transfer with minimal disruptions. This graph illustrates how TCP effectively manages varying network conditions to optimize data delivery.


Round Trip Time for 10.0.2.15:40466 → 128.119.245.12:80
Round Trip Time for 10.0.2.15:40466 → 128.119.245.12:80
eth0
Round Trip Time (ms)
0.4
0.35
0.3
0.25
0.2
0.15
0.1
0
1
2
3
4
5
Time (s)
Hover over the graph for details. → 34 pkts, 153 kB ← 112 pkts, 240 bytes
Type
Round Trip Time
Stream
1
Switch Direction
Mouse
drags
zooms
RTT By Sequence Number
Reset
Save As...
Close
Help


Round Trip Time for 10.0.2.15:40466 → 128.119.245.12:80
Round Trip Time for 10.0.2.15:40466 → 128.119.245.12:80
eth0
Round Trip Time (ms)
0.4
0.35
0.3
0.25
0.2
0.15
0.1
0
25000
50000
75000
100000
125000
150000
Sequence Number (B)
Hover over the graph for details. → 34 pkts, 153 kB ← 112 pkts, 240 bytes
Type
Round Trip Time
Stream
1
Switch Direction
Mouse
drags
zooms
RTT By Sequence Number
Reset
Save As...
Close
Help

B. TCP Packet Loss and Congestion Control

<u>Question 9 [10 pts]:</u><u>Time-Sequence-Graph(Stevens)</u>: Select a TCP packet with len != 0 that has been transmitted by the client and use the Time-Sequence-Graph(Stevens) plotting tool to create a graph. Include a **screenshot** of this graph.

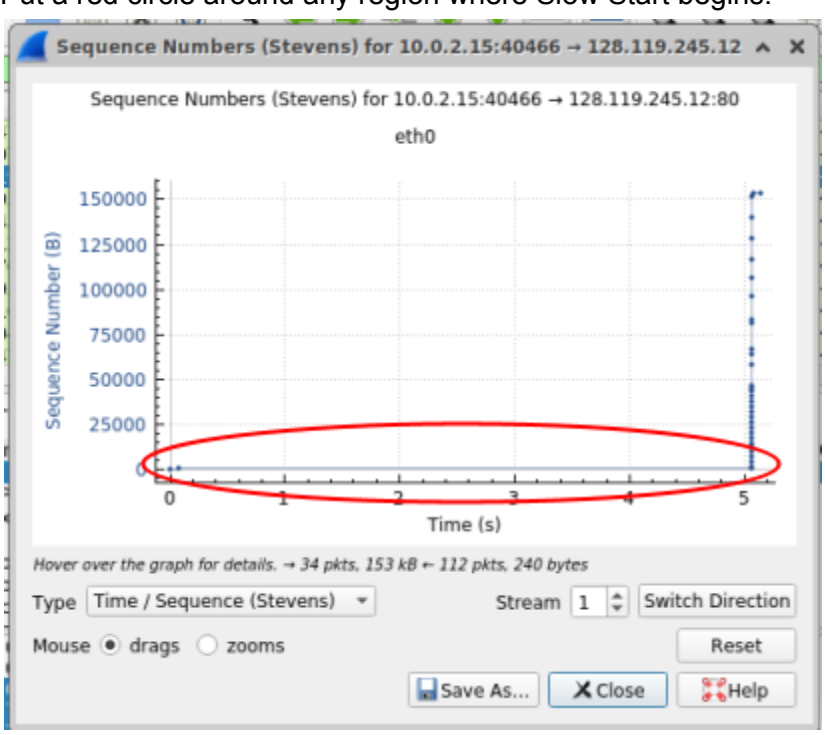
a. How many bytes are in the entire TCP segment you selected? (payload + header)
   2974 Bytes

b. Explain what is displayed on the x and y-axis in the graph. What does each dot represent?
   The x-axis represents the time in seconds, which is the elapsed time since the beginning of the TCP connection.

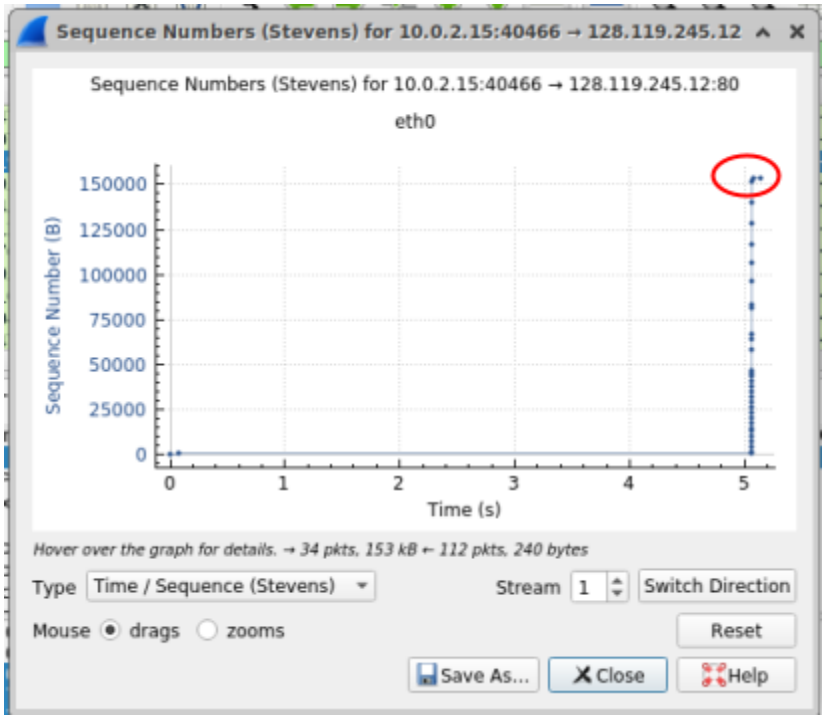
   The y-axis represents the TCP sequence numbers corresponding to the data being sent. Each sequence number indicates the byte offset of the first byte in the segment relative to the start of the connection.

   A single dot represents one TCP segment sent by the sender. The position of the dot indicates when the segment was sent (x-axis) and the starting byte offset (sequence number) of the data in that segment (y-axis). Together, these dots illustrate the flow of data over time in the connection.

c. Put a red circle around any region where Slow Start begins.



d. Are there any retransmitted segments in your graph? If so, circle them in red on the graph. Make sure your graph shows the <u>entire transfer and the lost segments, if any</u>.

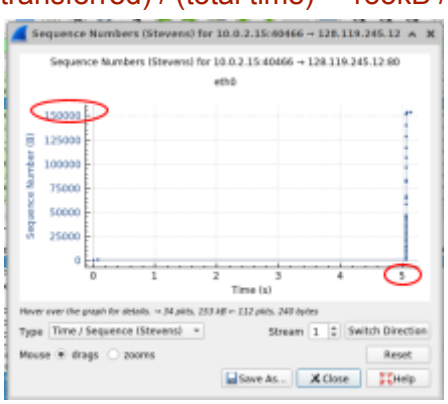


The horizontal line pattern indicates retransmitted segments, the same sata is being sent again after a timeout or lost acknowledgement. This is likely caused by a network congestion of packet loss during the final stages of data transfer.

<u>Question 10 [7pts]: Average Transmission Rate for File Upload</u>

Based on the Time-Sequence-Graph(stevens), what is the average transmission rate (over the duration of the file upload)? Explain how you calculated this value (your process) and support your calculation by highlighting it in your **screenshot**.

The highest sequence number is around 153kB. The average transmission rate = (total data transferred) / (total time) = 153kB / 5 sec = 30,600 bytes / second = 30.6 KBps.



Downloading a File from a Server (with packet loss)

Environment: Use the **VIRTUAL MACHINE** for these problems.

In this section, we will examine how loss affects a TCP connection. Loss will be simulated with the `tc qdisc` command to apply a loss rate to an incoming interface. We will observe TCP's response to packet loss.

No introduced losses:

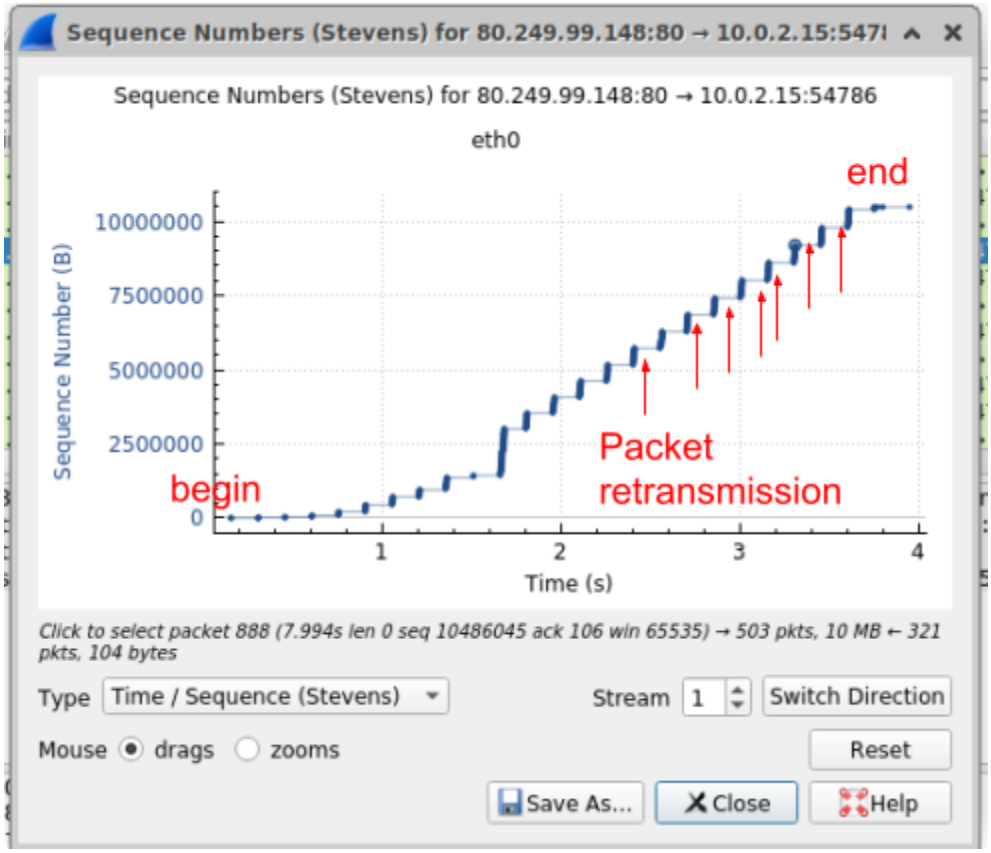
Question 11 [15 pts]: In this problem we will not intentionally introduce any loss. Start Wireshark and use `curl` to download this 10MB file.

a. Is packet loss possible during this download?
   Yes, packet loss is possible during the download, even if no intentional loss is introduced. Loss can occur due to factors such as network congestion, buffer overflows, or unstable network conditions. TCP is designed to handle such losses through retransmissions, which can be observed in the graph as repeated sequence numbers or horizontal lines.

b. Select a TCP data segment received by your client and create TimeSequenceGraph(Stevens). Include a **screenshot** of the graph and highlight important areas of the graph such as transfer begin/end, retransmitted packets etc. Write a short summary of your observations.

   The graph illustrates a smooth and successful file transfer of 10 MB over approximately 4 seconds. The steady slope indicates consistent data transmission, with no significant packet loss or retransmissions observed. The absence of horizontal lines in the graph reflects that TCP effectively managed congestion and ensured reliability during the transfer. This smooth flow highlights the stability of the network conditions during the download.

   I did not mark ALL of the retransmissions because there were too many, but the small horizontal segments are where retransmissions appear as the same sequence number is benign re-sent again.



c. Average throughput: Choose 2 points on the TimeSequenceGraph in (b) and from those data points calculate the <u>average throughput</u>. Show your calculations and support your choice of the data points

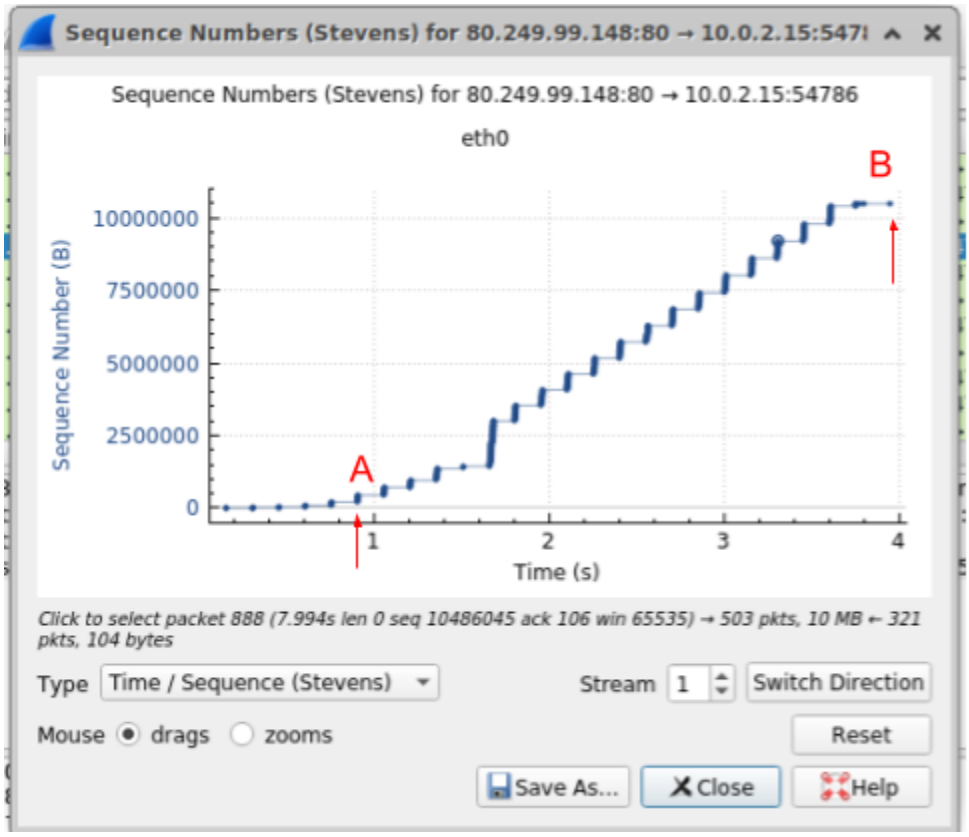
Point A: Time T_A and Sequence number S_A
Point B: Time T_B and Sequence number S_B

T_A = 0.9 seconds
T_B = 4 seconds

Data transferred = approximately 10 MB

Throughput = 10 MB / (4 - 0.9) = 3.23 MBps

Points A and B were chosen because they span a significant portion of the file transfer. Since there were no significant congestions, it seemed appropriate to include most of the transmission.



Introducing Loss:

Now we will simulate loss by using the command ***`tc qdisc`*** on an interface. When the command is first used, you must use ***add dev*** for the interface being changed. After adding the interface, use ***change dev*** to set the loss rate.

The following sequence of commands can be used to simulate 100% loss on the `eth0` interface:

- `sudo tc qdisc add dev eth0 root netem loss 0%`
- **Change loss to 100%**
  `sudo tc qdisc change dev eth0 root netem loss 100%`
- **Change loss back to 0%**
  `sudo tc qdisc change dev eth0 root netem loss 0%`

Really dismal – 100% loss event:

Read through this paragraph before starting:

- Start Wireshark
- Open 2 terminals and have commands typed and ready before you begin:
  - Terminal 1: Download the 10MB.zip file using curl

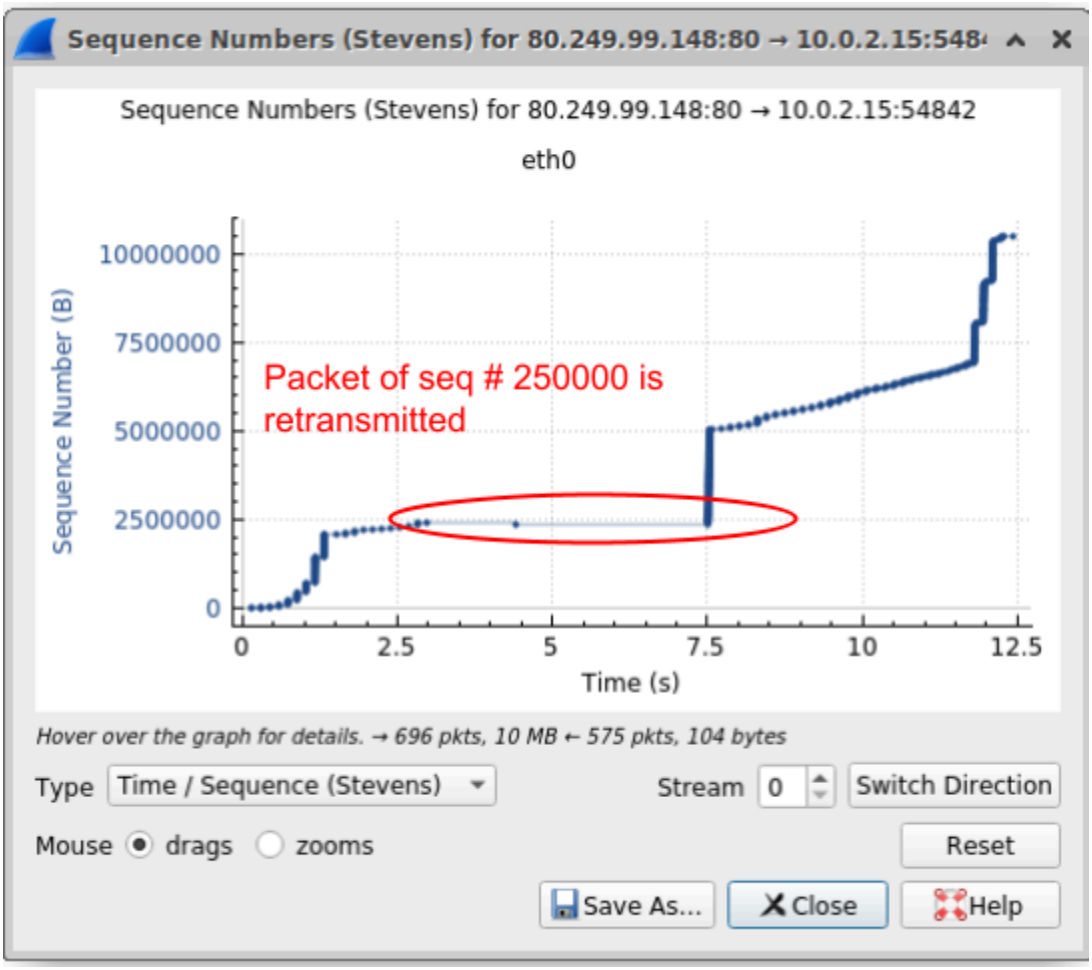
- Terminal 2: While the download is in progress, change loss to 100%. After a second, change loss back to 0%.

<u>Question 12 [10 pts]:</u> Answer the following questions by examining the Wireshark trace. Find a TCP data segment received by your client and create a TimeSequenceGraph(Stevens) graph with this packet selected.

a. At what time in your graph does packet loss begin? At what time does it end?
The packet loss begins at around 3 seconds. It ends around 7.5 seconds.

b. Search for an area of the graph where there is a retransmission. What Sequence Number is retransmitted?
Screenshot: Circle in red the region where 100% loss begins and ends in the graph. Identify the retransmitted packet (label it) in your screenshot.

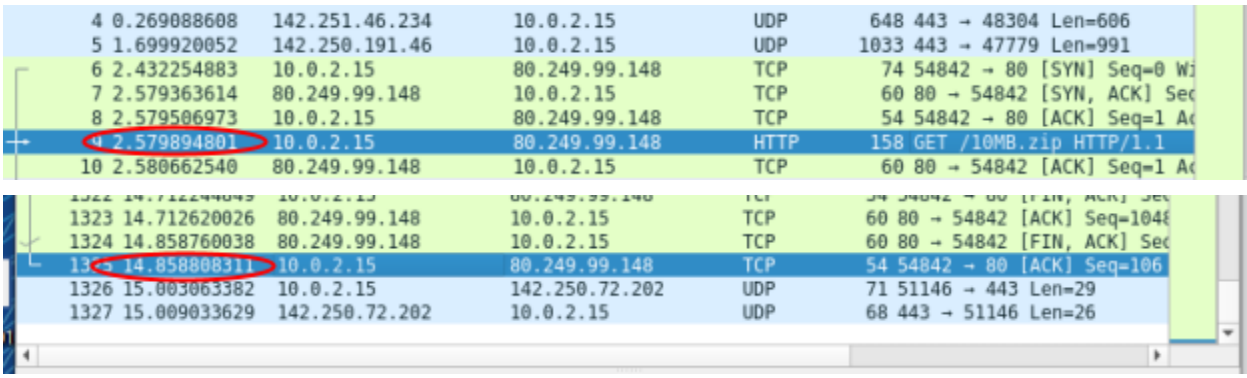
Sequence number of about 250000 is retransmitted.



c. Calculate the <u>average throughput</u> (consider the entire duration until the file transfer completes). Show your calculations and support with the **timestamped**

Wireshark capture by circling in red the first and last packet used for your calculations.

The average throughput is 10 MB / (14.8588 - 2.5798) = 0.8143985667 MBps



4	0.269088608	142.251.46.234	10.0.2.15	UDP	648 443 → 48304 Len=606
5	1.699920052	142.250.191.46	10.0.2.15	UDP	1033 443 → 47779 Len=991
6	2.432254883	10.0.2.15	80.249.99.148	TCP	74 54842 → 80 [SYN] Seq=0 Wi
7	2.579363614	80.249.99.148	10.0.2.15	TCP	60 80 → 54842 [SYN, ACK] Sec
8	2.579506973	10.0.2.15	80.249.99.148	TCP	54 54842 → 80 [ACK] Seq=1 Ac
9	2.579894801	10.0.2.15	80.249.99.148	HTTP	158 GET /10MB.zip HTTP/1.1
10	2.580662540	80.249.99.148	10.0.2.15	TCP	60 80 → 54842 [ACK] Seq=1 Ac

1323	14.712620026	80.249.99.148	10.0.2.15	TCP	60 80 → 54842 [ACK] Seq=1048
1324	14.858760038	80.249.99.148	10.0.2.15	TCP	60 80 → 54842 [FIN, ACK] Sec
1325	14.858808311	10.0.2.15	80.249.99.148	TCP	54 54842 → 80 [ACK] Seq=106
1326	15.003063382	10.0.2.15	142.250.72.202	UDP	71 51146 → 443 Len=29
1327	15.009033629	142.250.72.202	10.0.2.15	UDP	68 443 → 51146 Len=26

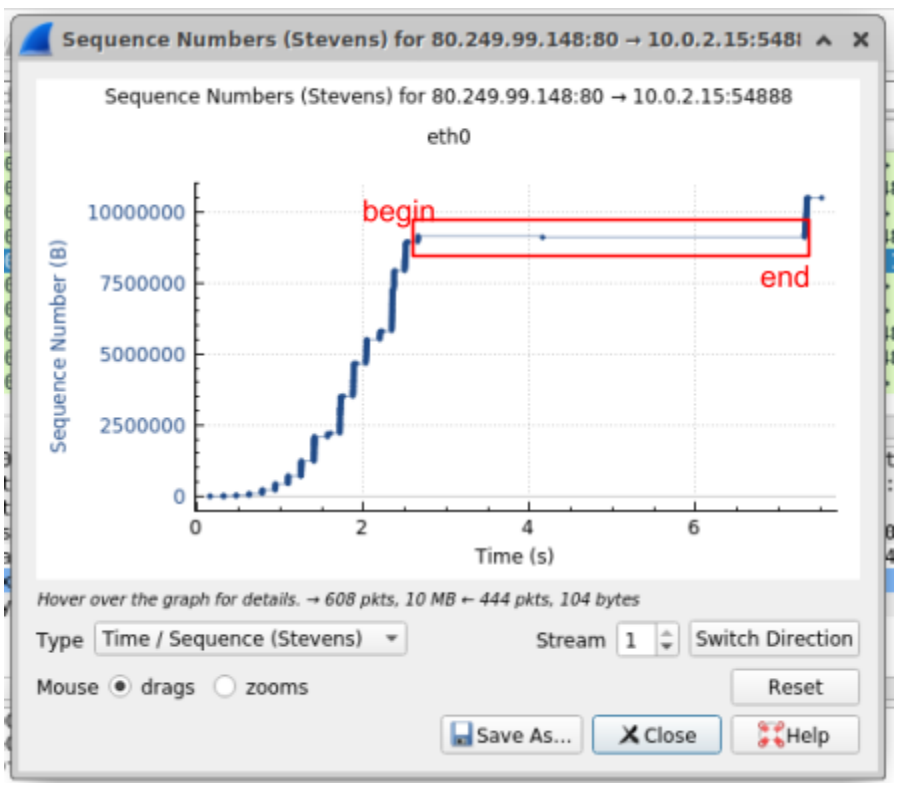
Not so bad? – 20% loss event:

Question 13 [12 pts]: Restart Wireshark and retry the experiment, this time with 20% packet loss. Introduce 20% loss and then quickly return to loss 0%. Find a TCP data segment received by your client and create a TimeSequenceGraph(Stevens) graph. Note: **you can use the Zoom function** to zoom in on an area of the graph to understand what is happening.

Attach a **screenshot** that includes markup for (a) and (c). It is ok if you want to use two screenshots.

a. At what time does the region which is experiencing 20% loss begin and end? In your **screenshot**, put a red box around this region.

It begins around 2.5 seconds and ends at around 7.2 seconds.

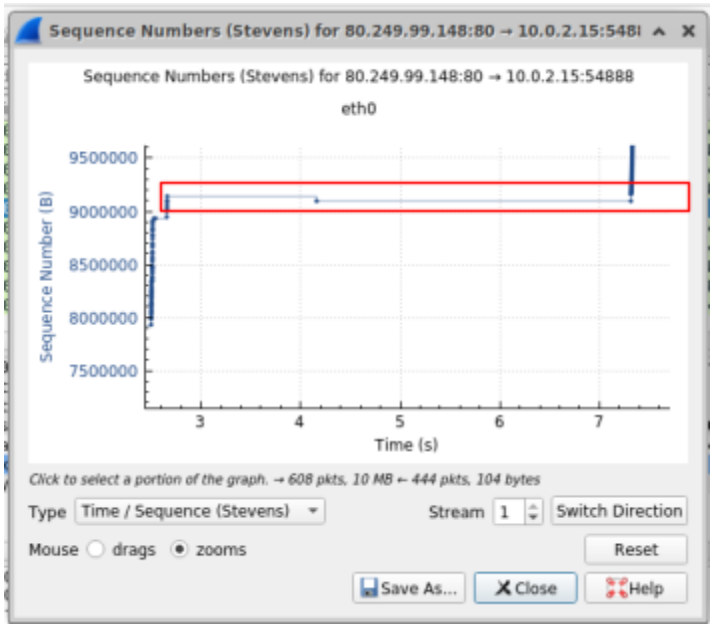


b. Examine in detail (zoom when necessary) the areas in the graph at the beginning of the data transfer and after the period of loss. What do you notice about these regions in the graph with respect to the TCP congestion control algorithms?

At the beginning of the data transfer, the graph shows a steep slope, indicating TCP's slow start phase where the congestion window (cwnd) grows exponentially as packets are acknowledged, allowing rapid utilization of available bandwidth. After the period of 20% packet loss (marked by horizontal segments), the graph reflects TCP's transition to

congestion avoidance or recovery, with a slower, linear increase in sequence numbers as TCP cautiously adjusts its sending rate. Retransmissions are evident as horizontal lines followed by upward movement, showing lost packets being resent. This behavior aligns with TCP's congestion control mechanisms, balancing throughput and reliability by reducing the transmission rate during congestion and gradually recovering afterward.

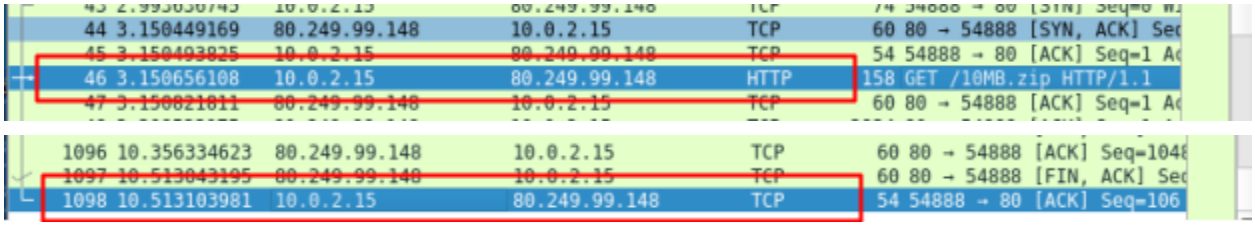
c. How do you expect to determine where there has been a retransmission? Search for a retransmission in the graph. What Sequence Number is retransmitted? Circle and label a retransmitted packet in your **screenshot**.



The sequence number of around 9000000 bytes experiences a retransmission, where the graph flattens for a period before resuming upward progression. This retransmission occurs after a period of packet loss or delay in acknowledgment.

d. Calculate the average throughput (consider the entire duration until the file transfer completes). Show your calculations and support with your **timestamped Wireshark capture** by **circling in red** the first and last packet used for your calculations.

The average throughput is 10 MB / (10.513 - 3.151) = 1.3583265417 MBps



Comparison of Download Performance:

Question 14 [8 pts]:. Make a table comparing the loss rates in Q11, 12 and 13. Columns should show your throughput calculation and the total time it took to download the 10MB file.

Discuss your results – Do your results make sense to you? What does the difference in throughput tell you about the operation of TCP during periods of congestion or lossy links?

	No Loss	100% Loss	20% Loss
Avg Throughput (MBps)	3.23	0.81	1.36
Total Time (s)	3.1	12.279	7.362

The table shows a clear relationship between packet loss and the performance of TCP. With no loss, the average throughput is highest at 3.23 MBps, and the file download completes in just 3.1 seconds. As packet loss increases, the throughput significantly decreases, with 20% loss resulting in a throughput of 1.36 MBps and a download time of 7.362 seconds. In the extreme case of 100% loss, throughput drops drastically to 0.81 MBps, and the download takes over 12 seconds.

This behavior aligns with TCP's congestion control mechanisms. When packet loss occurs, TCP reduces its transmission rate by shrinking the congestion window and retransmitting lost packets, leading to lower throughput and longer download times. The results demonstrate how TCP prioritizes reliability by adapting to lossy links at the cost of reduced performance. This tradeoff ensures data integrity but highlights the significant impact of packet loss on transfer efficiency.